FORMADORES { IT }

Kubernetes

7. Persistencia

Índice

PersistentVolumes

Tipos de volúmenes con soporte

PersistentVolumeClaims

StorageClass

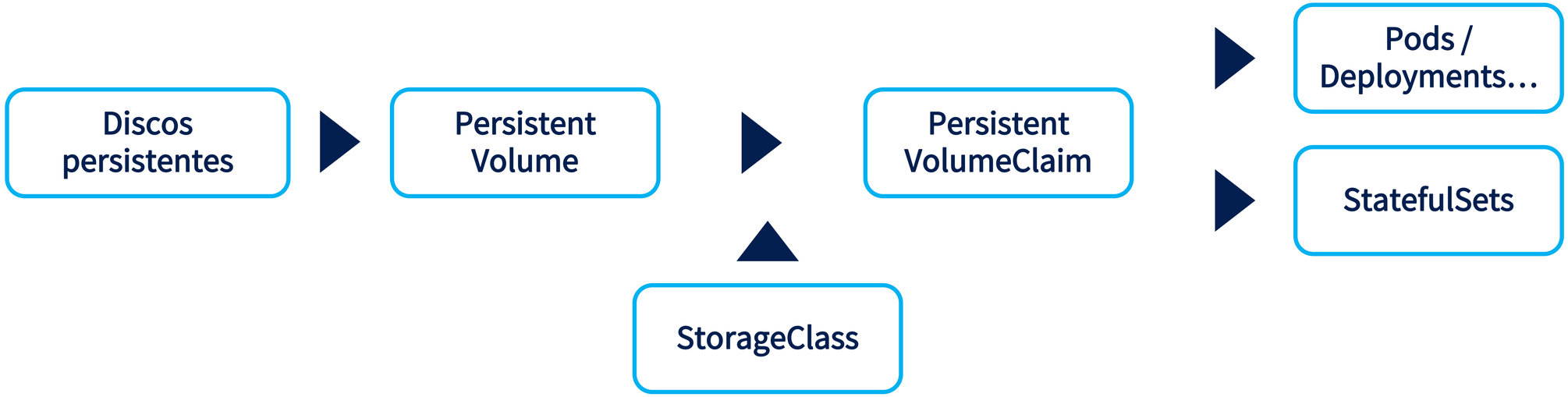
Montar volúmenes en pods

Volúmenes en StatefulSets

Objetivos

- Conoces los objetos PersistentVolumes y PersistentVolumeClaims
- Ver distintas opciones para obtener discos persistentes
- Configurar distintos tipos de volúmenes con StorageClasses
- Ver cómo configurar StatefulSets para que sean persistentes

Mapa Conceptual



Persistencia en Kubernetes

Kubernetes es una plataforma para todo. Ya hemos visto cómo configurar aspectos como la alta disponibilidad, la creación de IPs, de direcciones DNS, y de balanceadores de carga, el soporte para procesos de sistema y trabajos programados, etc.

Otro de los aspectos del despliegue de una aplicación que normalmente es complicado de tratar y que Kubernetes automatiza es la persistencia de datos.

Kubernetes nos permite configurar y utilizar cualquier tipo de disco persistente que tengamos en nuestros sistemas para utilizarlo en nuestros contenedores de forma muy simple.

Persistencia en Kubernetes

Una vez conozcamos los objetos que vamos a ver en este módulo y los configuremos correctamente, Kubernetes se encargará de todo lo referente a la gestión de estos discos o volúmenes, desde asignarlos al nodo específico en el que hacen falta, hasta reutilizarlos cuando pueden ser liberados o cambiar su tamaño de forma dinámica.

Además cuando estemos trabajando con Kubernetes en plataformas cloud (públicas y privadas), también se encargará de crear y gestionar estos discos en la nube de forma automática, con soporte para todo tipo de nubes, incluidas las más populares.

PersistentVolumes

Un **disco** de almacenamiento persistente en Kubernetes se representa con el objeto **PersistentVolume**. Es el objeto fundamental con el que trabajaremos este módulo, equivalente al pod para cargas de trabajo.

Un PersistentVolume puede representar un disco, o un directorio en un espacio compartido, o un volumen en un cloud, etc. Cualquier objeto en el que un pod pueda almacenar información y que esta persista cuando el pod se reinicie o cambie de nodo.

PersistentVolumes

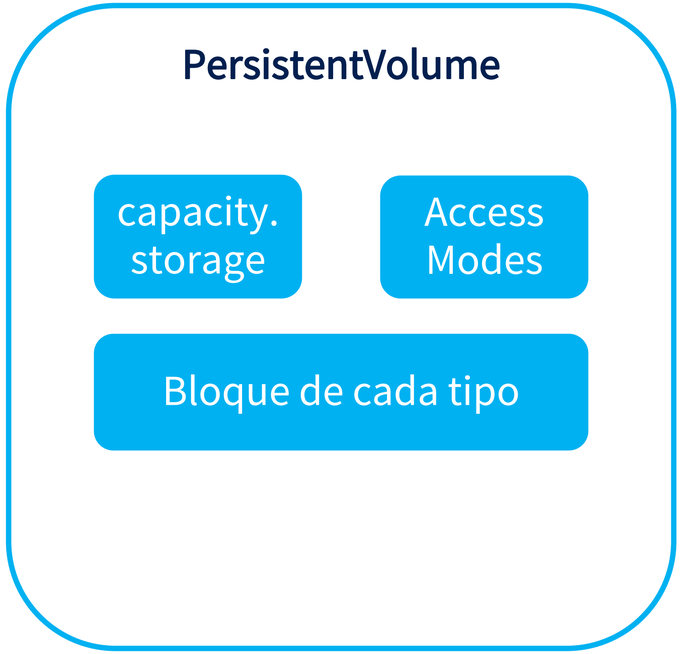
Estos son todos los sistemas de almacenamiento que soporta el objeto PersistentVolume:

- GCEPersistentDisk
- AWSElasticBlockStore
- AzureFile
- AzureDisk
- CSI
- FC (Fibre Channel)
- FlexVolume
- Flocker
- NFS
- iSCSI
- RBD (Ceph Block Device)
- CephFS
- Cinder (OpenStack block storage)
- Glusterfs
- VsphereVolume
- Quobyte Volumes
- HostPath Portworx Volumes
- ScaleIO Volumes
- StorageOS

PersistentVolumes

```
apiVersion: v1
kind: PersistentVolume
metadata:
    name: <Nombre>
spec:
    capacity:
      storage: <Tamaño en Gi>
    accessModes:
    - ReadWriteOnce

    <... Bloque específico para cada tipo...>
```



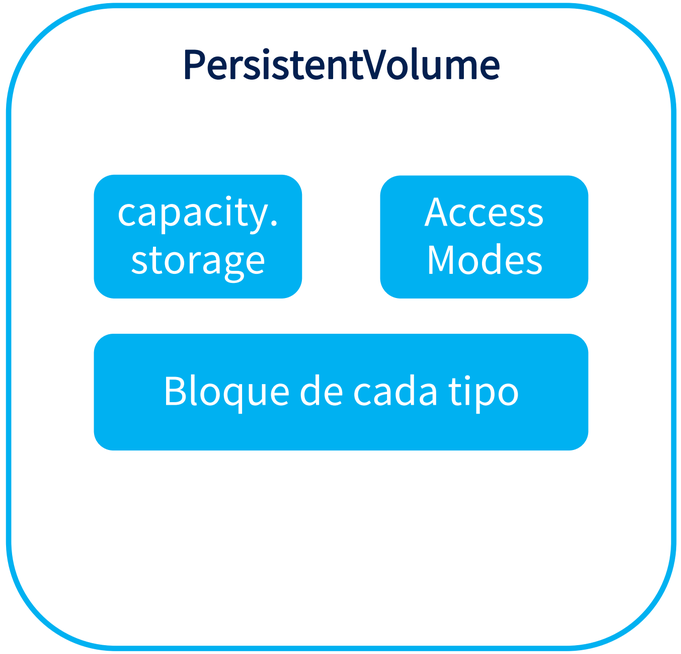
PersistentVolumes

Los campos obligatorios son 3:

1. capacity.storage: El tamaño que tendrá el volumen (normalmente en Gi)
2. AccessModes: Lista con los tipos de acceso permitidos. Estos pueden ser:
   1. ReadWriteOnce
   2. ReadOnlyMany
   3. ReadWriteMany

   No todos AccessMode están disponibles en todos los tipos de volúmenes

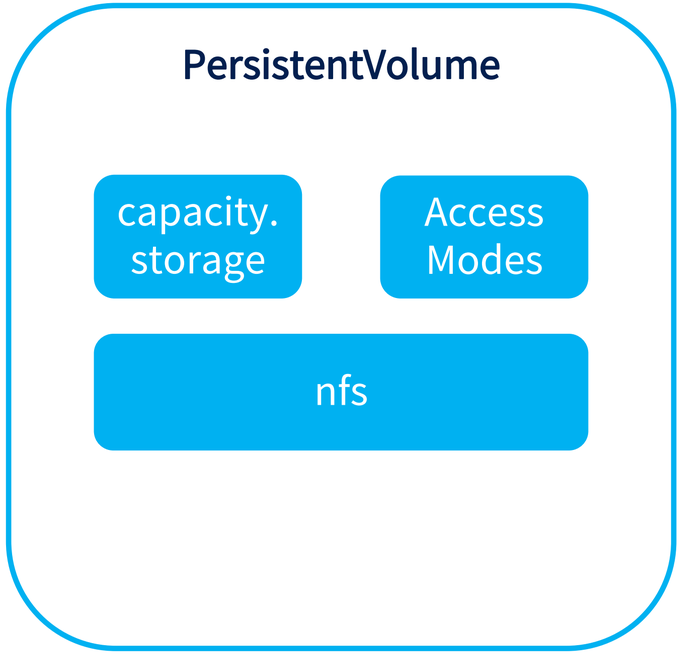
3. Un bloque específico del tipo de volumen



PersistentVolumes

Por ejemplo, para crear un volumen en un servidor NFS, el bloque específico sería:

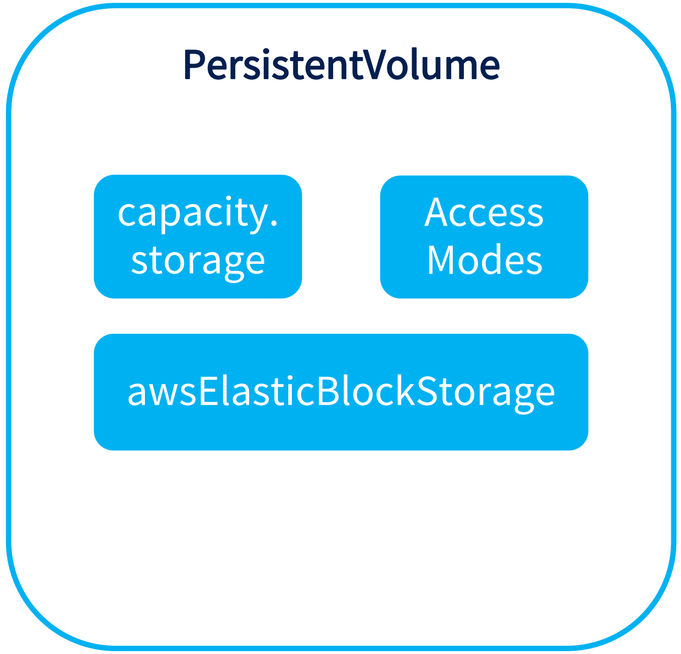
```
nfs:
    path: <Path dentro del servidor NFS>
    server: <Dirección IP del servidor>
```



PersistentVolumes

Para un volumen que represente un disco EBS en Amazon Web Services:

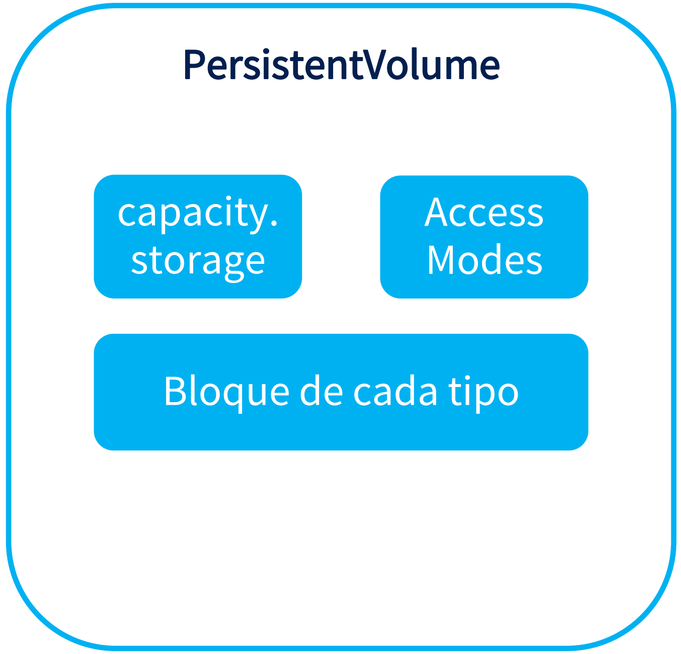
```
awsElasticBlockStore:
    volumeID: <ID del disco de EBS en AWS>
    fsType: <Tipo de sistema de archivos, normalmente ext4>
```



PersistentVolumes

Otros parámetros que podemos configurar en el PV son:

- volumeMode: Filesystem o block, dependiendo de si queremos montarlo en el pod como un disco con sistema de archivos como ext4 o directamente el disco como dispositivo.
- storageClassName: Hablaremos de ellas más adelante
- persistentVolumeReclaimPolicy: Retain, Recycle o Delete. Qué hacer cuando un volumen deja de ser reclamado.



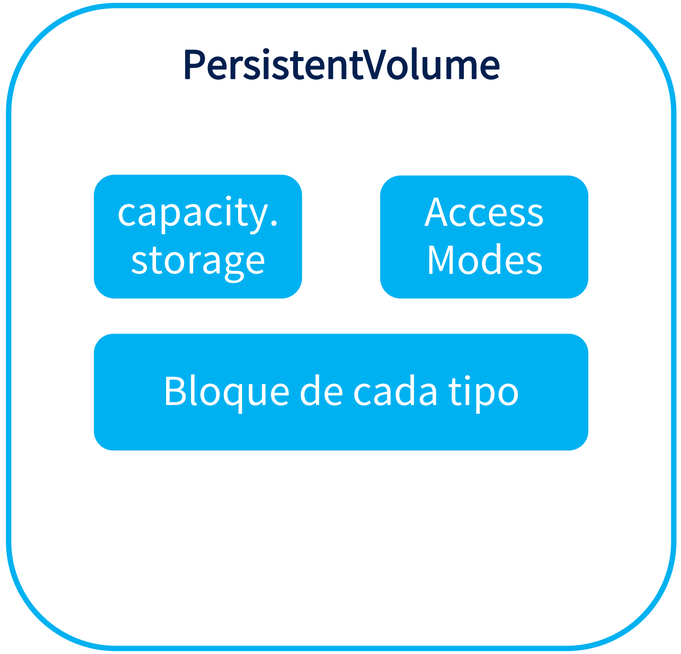
PersistentVolumes

Vamos a crear en nuestro entorno de prueba un volumen de 3Gi.

En Katacoda no tenemos disponible ningún sistema de almacenamiento por lo que crearemos un volumen del tipo hostPath.

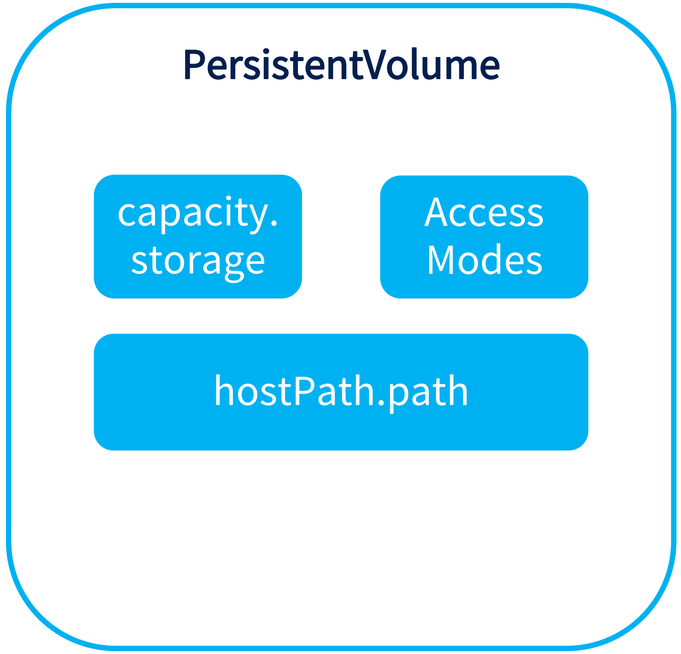
Este tipo de volumen representa simplemente un directorio en un nodo. No es posible utilizarlo en clústeres multimodo ya que no se puede mover de un nodo a otro. Como en Katacoda los pods sólo pueden ser asignados al nodo node01, nos valdrá.

El tipo hostPath está pensado solamente para entornos de pruebas.



PersistentVolumes

```
apiVersion: v1
kind: PersistentVolume
metadata:
    name: my-volume
spec:
    capacity:
      storage: 3Gi
    accessModes:
    - ReadWriteOnce
    hostPath:
      path: "/mnt/my-volume"
```



PersistentVolumes

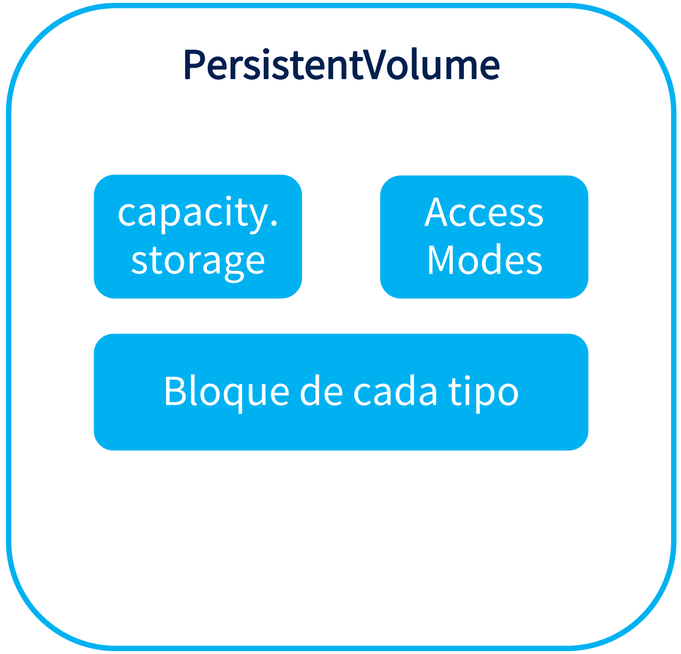
El volumen my-volumen representa el directorio /mnt/my-volumen en node01, que puede tomar un tamaño de hasta 3Gi.

Podemos listar los persistentVolumes con

```
kubectl get pv
```

Y ver los datos del que acabamos de crear con

```
kubectl describe pv my-volume
```



PersistentVolumeClaim

Ahora este volumen está disponible para ser utilizado en Kubernetes.

Pero en Kubernetes un volumen específico no está pensado para un pod en particular, sino que Kubernetes trata los persistentVolumes como un pool de discos para utilizarlos cuando alguien los reclame. Algo así como la memoria y CPU en los nodos, están libre hasta que alguien hace un request.

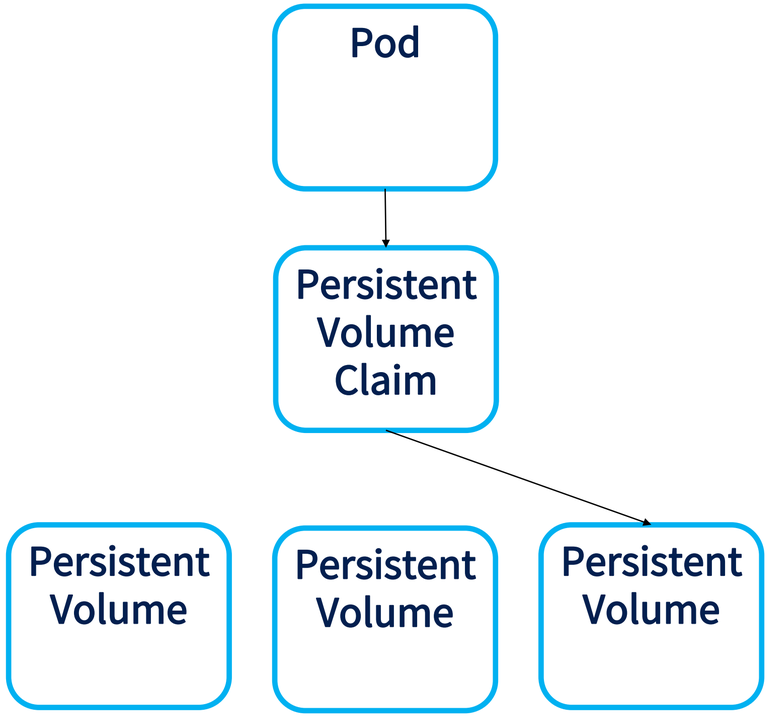
Para reclamar un disco que podamos montar en pods, antes tenemos que crear un persistentVolumeClaim.

PersistentVolumeClaim

Un persistentVolumeClaim es una petición para un persistentVolume pueda ser utilizado. Una vez el claim tiene asignado un persistentVolume en concreto, este puede ser montado como volumen en un pod.

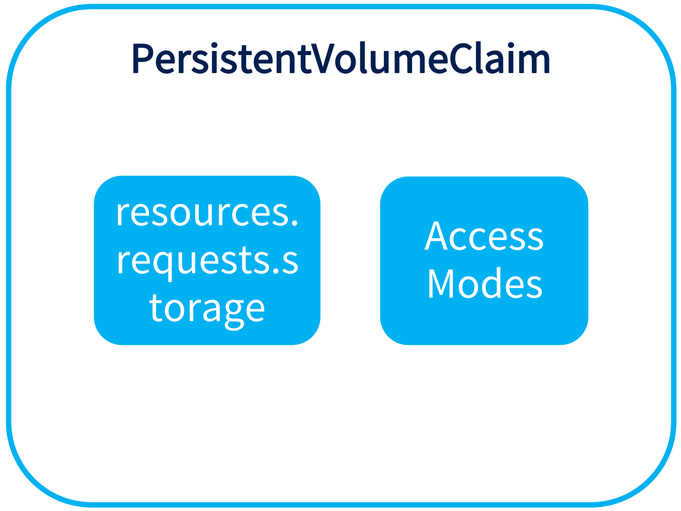
En resumen, un persistentVolumeClaim es lo que en realidad representa los datos persistentes.

Si se elimina el persistentVolumeClaim, el persistentVolume que tuviera asignado se libera para ser eliminado del todo o borrado para reutilizarse en el futuro por otro claim.



PersistentVolumeClaim

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nginx-pvc
spec:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 1Gi
```

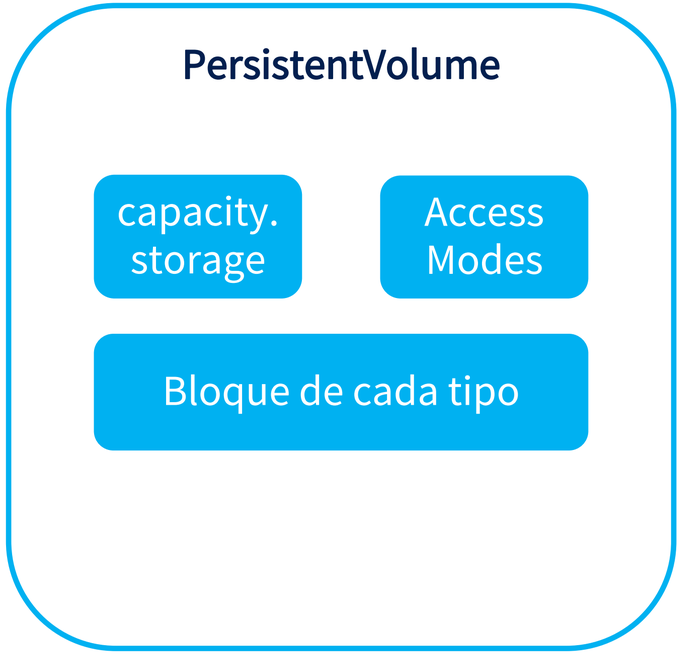


PersistentVolumeClaim

El persistentVolumeClaim contiene los requisitos que necesitamos para nuestra carga persistente.

En este caso, necesitamos un disco como mínimo con acceso ReadWriteOnce y con tamaño mínimo de 1Gi

Kubernetes ahora recorrerá automáticamente todos los persistentVolumes que existan y asignará uno que tenga como mínimo esos requisitos.



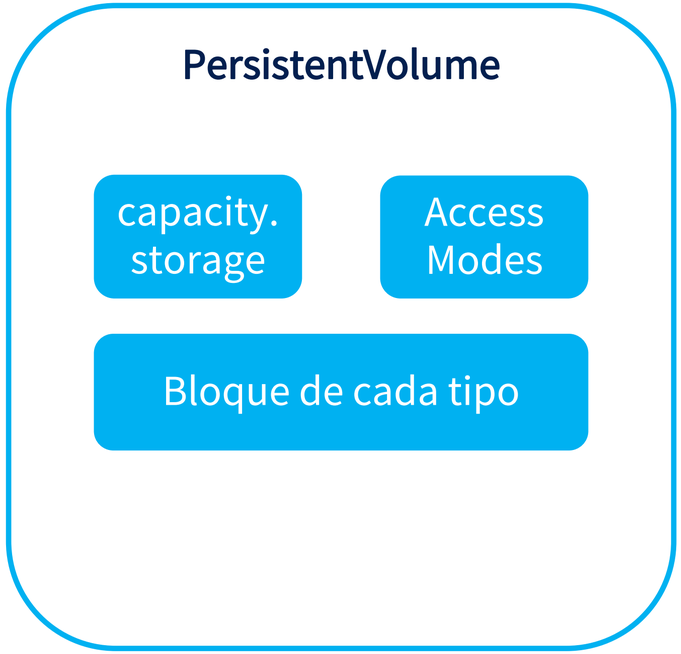
PersistentVolumeClaim

Podemos listar los persistentVolumeClaims con:

```
kubectl get pvc
```

Vemos que está en estado Bound ya que le han asignado el disco my-volume que creamos anteriormente.

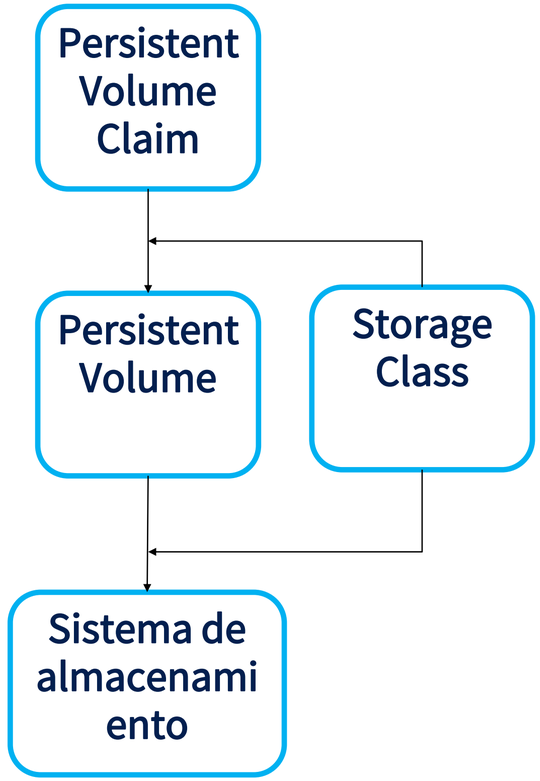
Si listamos los persistentVolumes, veremos que efectivamente el persistentVolume ha pasado de estado Available a Bound.



storageClasses

Normalmente los persistentVolumes no se crean manualmente. Kubernetes tiene controladores que se conectan a los sistemas de almacenamiento más utilizados (tanto en nube pública como privada), y crea automáticamente el disco y el correspondiente PV para cada PVC.

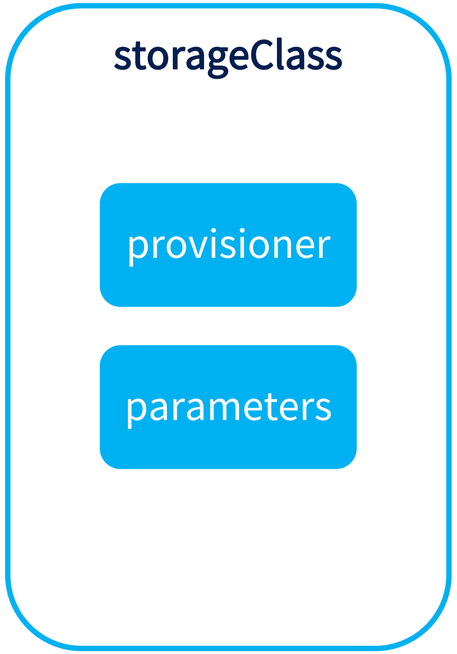
Esto se consigue mediante el objeto storageClass. StorageClass representa un tipo concreto de almacenamiento. Puede representar un sistema con un tipo de disco en concreto (SSD vs HD), con una velocidad de disco determinada, etc.



storageClasses

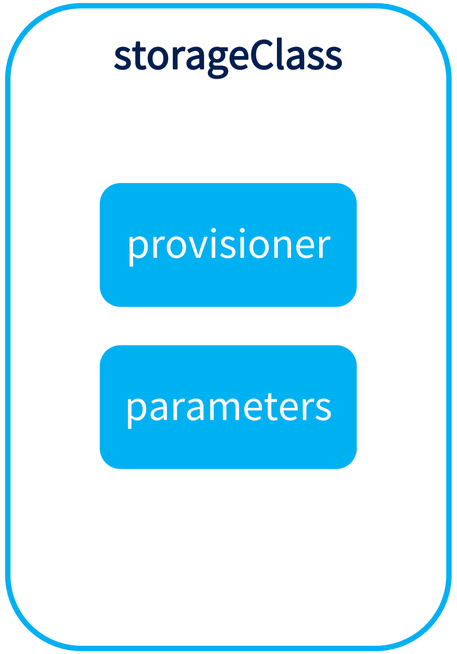
Una storageClass se compone de todos estos parámetros y del campo privisioner. Provisioner representa el sistema al que se conectará la storageClass para crear los discos de forma dinámica. Estos son los valores que puede tomar:

- AWSElasticBlockStore
- AzureFile
- AzureDisk
- CephFS
- Cinder
- FC
- FlexVolume
- Flocker
- GCEPersistentDisk
- Glusterfs
- iSCSI
- Quobyte
- NFS
- RBD
- VsphereVolume
- PortworxVolume
- ScaleIO
- StorageOS
- Local



storageClasses

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: test
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp2
```



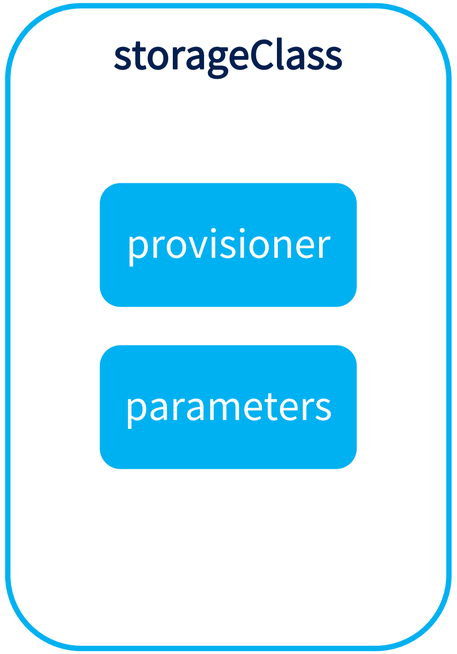
Esta storageClass crearía un disco EBS en AWS, de tipo gp2 (SSD), para todos los PVC que estén asignados a ella.

storageClasses

El campo parameters es específico de cada provisioner.

Otros campos interesantes de storageClass son:

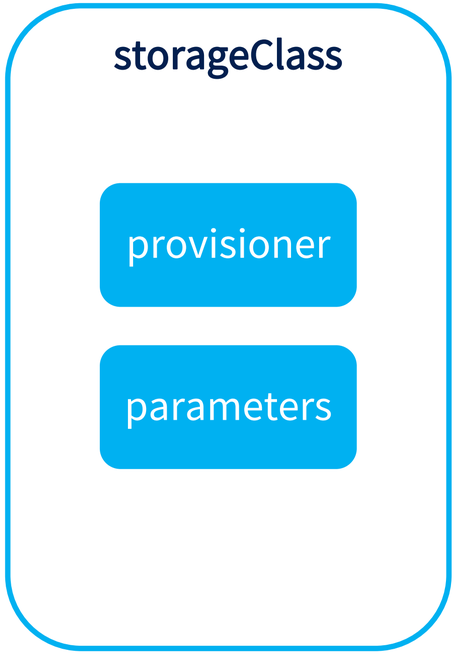
- reclaimPolicy: retain o delete. Es la política para tratar los discos cuando se elimina el PVC al que estaban asignados.
- volumeBindingMode : true o false. Quizás el más útil. Si es true, Kubernetes espera a que el PVC esté montado en un pod para crear el disco físico.
- allowVolumeExpansion: Algunos provisiones permiten expandir los discos automáticamente si el requisito del PVC es aumentado.



storageClasses

En el entorno de prueba no es posible comprobar esta funcionalidad, pero podemos crear una storageClass de tipo local para tener este objeto en nuestro clúster:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: manual-sto
provisioner: kubernetes.io/no-provisioner
```



storageClasses

Podemos listar las storageClasses del clúster con el comando:

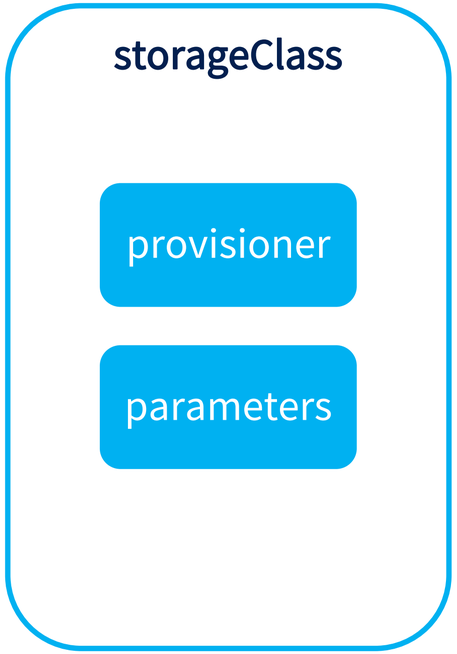
```
kubectl get sc
```

Para asignar un PVC a una storageClass, simplemente tenemos que añadir este campo en su spec:

```
storageClassName: manual-storage
```

Nosotros también tenemos que añadir ese parámetro a la definición del PV, ya que sino no podrá se asignado a PVCs de esa storageClass.

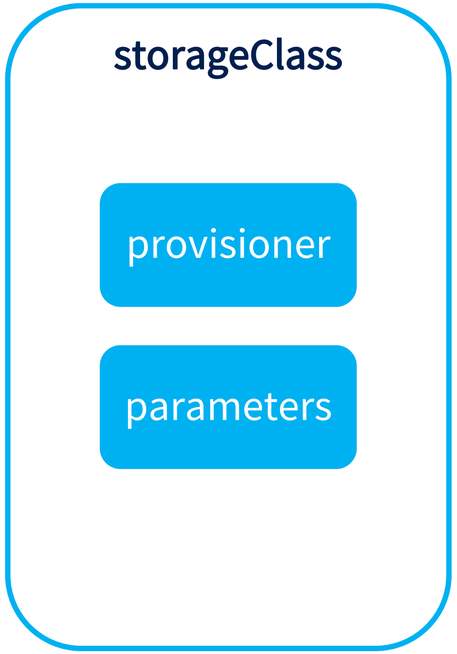
Los PVs creados automáticamente ya contienen ese campo en su spec.



storageClasses

El flujo típico de trabajo en Kubernetes es utilizar una storageClass con un provisioner que nos cree automáticamente los PVs y los discos en la nube. Casi nunca se crearán los PVs manualmente.

Es un comportamiento similar a los pods y los deployments, no se crean pods aislados sino que siempre son creados por algún objeto superior como un deployment.

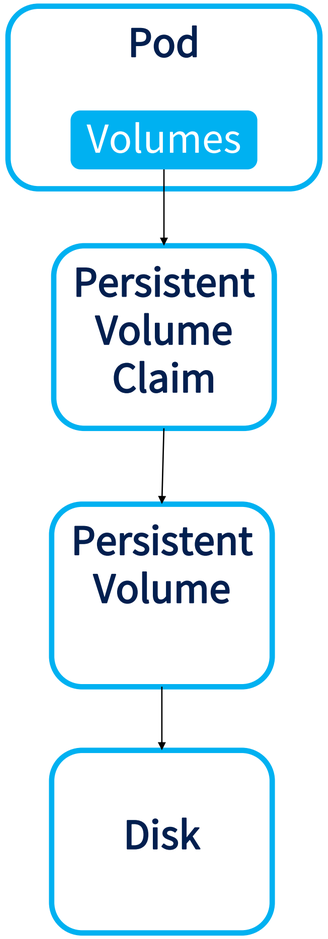


Volúmenes persistentes en pods

Una vez hemos conocido todo sobre cómo gestionar PVs, PVCs y storageClasses, podemos por fin montar estos discos en pods.

Simplemente hay que añadir el PVC como volumen en el spec del pod:

```
volumes:
        - name: data
          persistentVolumeClaim:
            claimName: my-data
```

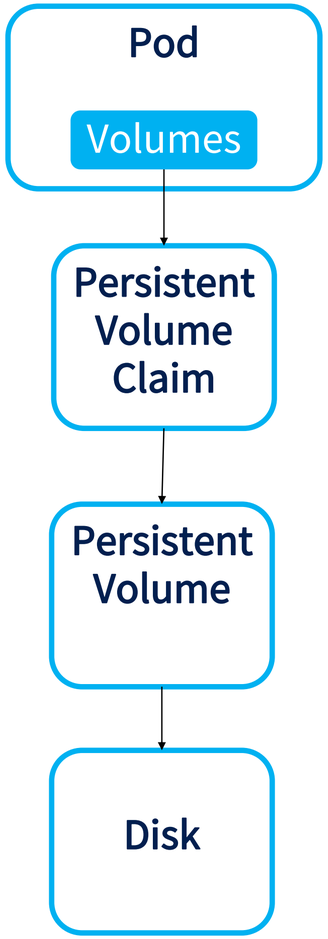


Volúmenes persistentes en pods

Y montarlo dentro de un contenedor en el apartado volumeMounts:

```
volumeMounts:
        - name: data
          mountPath: /root/my-data
```

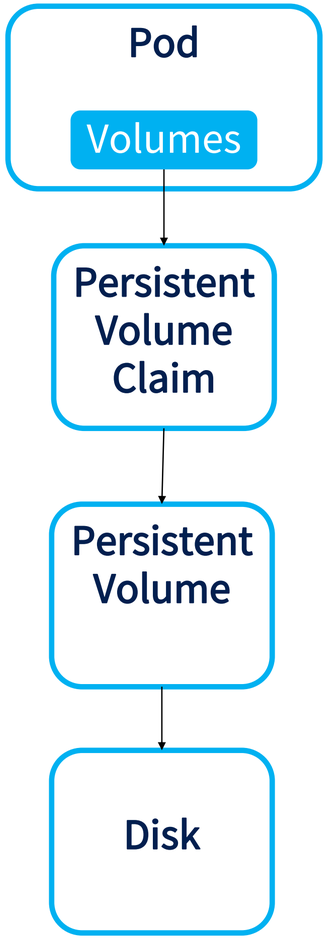
Con esto tendríamos el PV montado en el path /root/my-data. Todo lo que creemos dentro de ese path, queda almacenado en el PV, y si el pod se tiene que reiniciar o recrear, esa información se mantiene



StatefulSets

En un statefulSet cada pod tiene asignado su propio volumen y estos pueden escalar automáticamente. ¿Cómo podemos gestionar el escalado cuando necesitamos un PVC nuevo para cada pod?

Los statefulSets tienen un campo llamado volumeClaimTemplate, que nos permite definir unos parámetros para que se creen PVCs automáticamente para cada réplica.



StatefulSets

```
volumeClaimTemplates:
 - metadata:
     name: data
   spec:
     accessModes: [ "ReadWriteOnce" ]
     resources:
       requests:
         storage: 1Gi
```

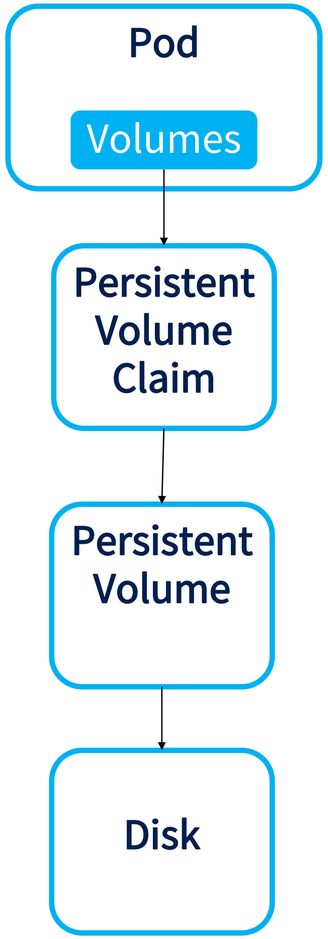
StatefulSets

Kubernetes creará automáticamente un PVC llamado

<Nombre StatefulSet>-<Nombre volumeClaimTemplate-<índice>

Cada vez que se cree un nuevo pod del statefulSet.

Además puede conservar este PVC para que se mantenga cuando escalamos y desescalamos el statefulSet



¡Fin del módulo!

En este módulo hemos visto todos los objetos relacionados con la persistencia de datos en Kubernetes:

- Los PersistentVolumes (PVs) representan un disco físico persistente de cualquier tipo
- Los PersistentVolumeClaims (PVCs) representa unos datos persistentes en Kubernetes que necesita ser asignado a un PV
- Las StorageClasses sirven para crear automáticamente los PVs y los discos físicos para cada nuevo PVC
- Los PVCs se pueden montar en pods dentro del campo volumes
- Los statefulSets tienen la capacidad de crear sus propios PVCs para cada nueva réplica.

Extras

Páginas de la documentación oficial:

PVs y PVCs
StorageClass
Configurar pods para persistencia
StatefulSets

¡Gracias!